CSE 6730 Checkpoint 1: Comprehensive PDE Numerical Methods Comparison for Option Pricing

Repository Link

Group 12: Pascal Bermeo Neumann, Sarang Deshpande, Michael Waltuch

April 3, 2026

Abstract

This project develops a unified framework for comparing numerical solution methods for option pricing PDEs across multiple financial models, validated through calibration to real market data. We implement and compare three classes of numerical methods: (1) explicit, implicit, and Crank-Nicolson finite difference schemes for the 1D Black-Scholes PDE, (2) four Alternating Direction Implicit schemes (Douglas, Craig-Sneyd, Modified Craig-Sneyd, and Hundsdorfer-Verwer) for the 2D Heston stochastic volatility PDE, and (3) IMEX schemes for jump-diffusion PIDEs. Each method is rigorously evaluated for stability, convergence, and computational efficiency. The project applies Levenberg-Marquardt optimization to calibrate Heston model parameters to OptionMetrics implied volatility surfaces, demonstrating how discretization choices affect both numerical accuracy and empirical model fit. Our implementation in Python provides a modular, well-documented codebase that serves as both a pedagogical tool for understanding PDE numerical methods and a practical framework for quantitative finance applications.

1 Introduction

In the world of quantitative finance, partial differential equations have been applied extensively to options pricing, with various numerical methods being introduced to handle certain challenges unique to specific stochastic options pricing models. The Black-Scholes model, despite its elegant closed-form solution, relies on the unrealistic assumption of constant volatility. More sophisticated models like Heston's stochastic volatility framework and Merton's jump-diffusion model address these limitations but introduce computational challenges that require specialized numerical techniques. From one-dimensional diffusion equations to two-dimensional PDEs with mixed derivatives and partial integro-differential equations with nonlocal jump operators, different models require increasingly

sophisticated numerical approaches rather than a single generalizable solution method.

Our contribution is a unified framework that implements and rigorously compares numerical methods across three important option pricing models, validated through calibration to real market data from OptionMetrics. Specifically, we implement three classes of finite difference schemes for the one-dimensional Black-Scholes PDE, four alternating direction implicit (ADI) schemes for the two-dimensional Heston PDE, and implicit-explicit (IMEX) schemes for jump-diffusion partial integro-differential equations. Each method is evaluated not only on theoretical metrics such as convergence order and stability, but also on practical considerations including computational efficiency and calibration quality when fitted to market-observed implied volatility surfaces.

This project is fundamentally a simulation study focused on numerical methods for partial differential equations. While we utilize real market data, our primary objective is not statistical analysis but rather the systematic comparison of PDE solution techniques. By implementing these methods in a modular Python framework and validating them against both analytical solutions and empirical data, we demonstrate the interplay between numerical accuracy and empirical model fit.

The remainder of this document is organized as follows. Section 2 describes the financial models and their associated PDEs. Section 3 presents the conceptual model and overall system architecture. Section 4 details the development platform and tools. Section 5 provides a comprehensive status update on implementation progress and outlines the division of labor for completing the remaining work.

2 System Description

This project implements and compares numerical solution methods for option pricing across three fundamental financial models: the Black-Scholes model for constant volatility, the Heston model for stochastic volatility, and the Merton jump-diffusion model for price discontinuities. Each model presents distinct computational challenges that require specialized numerical techniques.

2.1 Financial Models

2.1.1 Black-Scholes Model (1D)

The Black-Scholes model assumes the underlying asset price S follows geometric Brownian motion with constant volatility σ:

$$dS_t = rS_t\,dt + \sigma S_t\,dW_t \tag{1}$$

where r is the risk-free rate and W_t is a Wiener process. The option value $V(S,t)$ satisfies the parabolic PDE:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0 \tag{2}$$

The Black-Scholes model serves as our baseline. While it has a closed-form solution for European options, solving the PDE numerically provides a foundation for understanding finite difference methods and validating our implementation before tackling more complex models. The one-dimensional spatial domain and absence of mixed derivatives make it an ideal testbed for explicit, implicit, and Crank-Nicolson schemes.

2.1.2 Heston Model (2D)

The Heston model introduces stochastic volatility to capture the empirically observed volatility smile and skew in option prices. The asset price S and variance v evolve according to:

$$dS_t = rS_t\,dt + \sqrt{v_t}S_t\,dW_t^S \tag{3}$$

$$dv_t = \kappa(\theta - v_t)\,dt + \xi\sqrt{v_t}\,dW_t^v \tag{4}$$

where κ is the mean reversion speed, θ is the long-run variance, ξ is the volatility of volatility, and $dW_t^S dW_t^v = \rho\,dt$ with correlation ρ. The option value $V(S, v, t)$ satisfies the two-dimensional PDE:

$$\begin{aligned}\frac{\partial V}{\partial t} &= \frac{1}{2}vS^2\frac{\partial^2 V}{\partial S^2} + \rho\xi vS\frac{\partial^2 V}{\partial S\partial v} + \frac{1}{2}\xi^2 v\frac{\partial^2 V}{\partial v^2} \\ &\quad + rS\frac{\partial V}{\partial S} + \kappa(\theta - v)\frac{\partial V}{\partial v} - rV\end{aligned} \tag{5}$$

The key computational challenge is the mixed derivative term $\partial^2 V/\partial S\partial v$, which couples the two spatial directions. This term arises from the correlation ρ between asset returns and volatility changes. Standard implicit finite difference methods would require solving a dense $(N_S \times N_v) \times (N_S \times N_v)$ linear system at each time step, which is computationally prohibitive. Alternating Direction Implicit (ADI) schemes circumvent this by splitting the 2D problem into a sequence of 1D problems.

2.1.3 Merton Jump-Diffusion Model (1D PIDE)

The Merton model extends Black-Scholes by adding discrete jumps to the asset price dynamics:

$$dS_t = (r - \lambda m)S_t\,dt + \sigma S_t\,dW_t + S_t dJ_t \tag{6}$$

where J_t is a compound Poisson process with intensity λ and log-normal jump sizes. The option value satisfies a partial integro-differential equation (PIDE):

$$\begin{aligned}\frac{\partial V}{\partial t} &= \frac{1}{2}\sigma^2 S^2\frac{\partial^2 V}{\partial S^2} + (r - \lambda m)S\frac{\partial V}{\partial S} - (r + \lambda)V \\ &\quad + \lambda\int_{-\infty}^{\infty} V(Se^y, t)\nu(y)\,dy\end{aligned} \tag{7}$$

where $\nu(y)$ is the jump size density and $m = E[e^Y - 1]$ is the expected relative jump size.

The integral term represents the expected value contribution from jumps and creates a nonlocal coupling across the spatial grid. This term cannot be treated implicitly without constructing and inverting a dense matrix. The IMEX (Implicit-Explicit) approach treats the diffusion operator implicitly for stability while handling the integral operator explicitly to avoid dense linear systems.

2.2 Numerical Methods

2.2.1 Finite Difference Schemes for Black-Scholes

We implement three classical finite difference schemes for the one-dimensional Black-Scholes PDE:

- **Explicit scheme**: Forward Euler in time, central differences in space. Simple to implement but conditionally stable, requiring $\Delta t \leq \Delta S^2/(\sigma^2 S_{\max}^2)$.
- **Implicit scheme**: Backward Euler in time, central differences in space. Unconditionally stable but only first-order accurate in time. Requires solving a tridiagonal system at each time step.
- **Crank-Nicolson scheme**: Averages explicit and implicit schemes ($\theta = 0.5$). Second-order accurate in both time and space while maintaining unconditional stability. Also requires tridiagonal solves.

All three schemes discretize the spatial second derivative using central differences:

$$\frac{\partial^2 V}{\partial S^2} \approx \frac{V_{i+1} - 2V_i + V_{i-1}}{\Delta S^2} \tag{8}$$

This produces a tridiagonal system structure that can be solved efficiently using the Thomas algorithm in $O(N)$ operations.

2.2.2 ADI Schemes for Heston

Alternating Direction Implicit (ADI) methods decompose the two-dimensional problem into a sequence of one-dimensional sweeps. At each time step, the method alternates between treating the S-direction implicitly while keeping the v-direction explicit, and vice versa. We implement four ADI schemes from In 't Hout and Foulon [3]:

- **Douglas scheme**: The simplest ADI approach with two sweeps per time step. First-order accurate in time, $O(\Delta t, \Delta S^2, \Delta v^2)$. The mixed derivative term is treated explicitly using values from the previous time step, which can cause instability for large correlation $|\rho|$.
- **Craig-Sneyd scheme**: Adds two correction sweeps to achieve second-order temporal accuracy, $O(\Delta t^2, \Delta S^2, \Delta v^2)$. The correction steps update the mixed derivative using intermediate values, significantly improving stability for strong correlation.

- **Modified Craig-Sneyd (MCS) scheme**: Modifies the treatment of the mixed derivative by applying half in the first sweep and half in the second sweep, rather than applying all of it initially. This more balanced approach enhances stability, particularly for large time steps.
- **Hundsdorfer-Verwer (HV) scheme**: Uses optimally weighted splitting of operators with parameters γ, α, and β chosen to minimize error constants. Achieves the best accuracy among ADI schemes at the cost of slightly increased complexity.

Each directional sweep requires solving N tridiagonal systems of size M, yielding $O(NM)$ complexity per sweep compared to $O((NM)^3)$ for a full implicit 2D scheme. The key algorithmic difference between schemes lies in how they handle the mixed derivative term $\rho\xi vS\partial^2 V/\partial S\partial v$.

2.2.3 IMEX Schemes for Jump-Diffusion

The Implicit-Explicit (IMEX) time-stepping method splits the PIDE operator into local (differential) and nonlocal (integral) components:

$$\frac{\partial V}{\partial t} = \mathcal{L}_{\text{diff}}(V) + \mathcal{I}_{\text{jump}}(V) \tag{9}$$

where $\mathcal{L}_{\text{diff}}$ contains the diffusion and convection terms, and $\mathcal{I}_{\text{jump}}$ is the integral operator.

The IMEX scheme as developed by Cont and Voltchkova [1] treats the diffusion operator implicitly for unconditional stability while evaluating the jump integral explicitly:

$$(I - \Delta t\mathcal{L}_{\text{diff}})V^{n+1} = V^n + \Delta t\mathcal{I}_{\text{jump}}(V^n) \tag{10}$$

The integral term is computed via numerical quadrature (typically Gaussian quadrature or adaptive Simpson's rule) over the jump size distribution. This approach avoids the dense linear systems that would arise from implicit treatment of the integral while maintaining the stability benefits of implicit time-stepping for the diffusion part.

2.3 Calibration Framework

Model parameters are estimated by minimizing the sum of squared differences between model-implied and market-observed option prices. For the Heston model, we calibrate the five parameters $\{\kappa, \theta, \xi, \rho, v_0\}$ to match the implied volatility surface. The objective function is:

$$\min_{\theta} \sum_{i=1}^{N} w_i \left(V^i_{\text{market}} - V^i_{\text{model}}(\theta)\right)^2 \tag{11}$$

where w_i are weights (typically inverse of bid-ask spread or vega) and the sum is over N liquid options across various strikes and maturities.

We employ the Levenberg-Marquardt algorithm, which combines gradient descent with Gauss-Newton updates. Following Cui et al. [2], we compute gradients of option prices with respect to model parameters numerically via finite differences on the PDE solution. This approach is significantly faster than re-computing the entire PDE solution for each parameter perturbation.

3 Conceptual Model

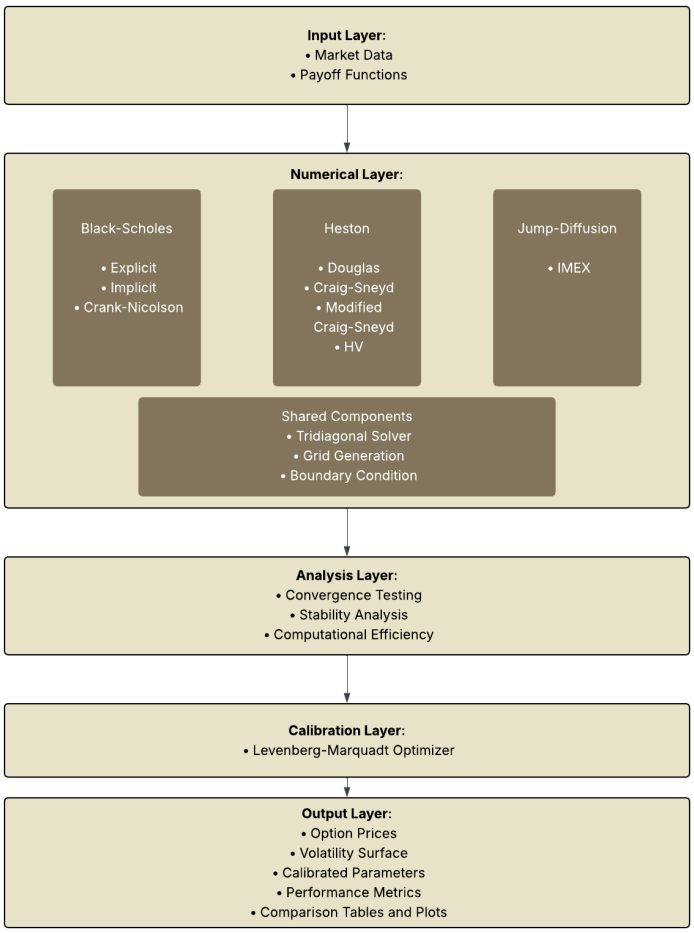

Figure 1: Conceptual Model

Key considerations when designing the conceptual model were modularity and re-usability. Each component should be independent and testable, and for any repetitive processes, we can have it be accessible anywhere in the repo.

4 Development Platform

4.1 Data

The data that will be used for the calibration of the different models was sourced from the OptionMetrics database offered by Wharton Research Data Services (WRDS). Accessing the data requires an API key that needs to be approved, but Sarang already had a key so to make sure everyone could access the data, the processed data is stored in the data folder in the repository.

WRDS has a Python API that allows for data to be pulled using Python scripts that run customizable queries. Relevant data for calibration includes option data (expiry date, strike price, implied volatility, etc.), stock data (date, price), interest rates data (date, risk-free interest rate). The various tables were merged on date and cleaned for data type issues. Features were also added, such as the spread between bid and offer prices. The data was finally filtered based on constraints for trading volume, volatility, etc. so that the data could properly be used for calibration. Parameters such as which stock to pull data for, or the dates for analysis, or filtering thresholds can all be adjusted in the public configuration file.

4.2 Platforms, Libraries, and Solvers

For this project, our group is working in Python 3.13 as our team is primarily proficient in Python. So far, the core libraries used have been NumPy, Pandas, Matplotlib, and WRDS. All numerical solvers have been implemented by our team, as we are not relying on any open-source solvers at this moment.

5 Project Status and Progress

5.1 Current Status

Currently, our team has developed a script to pull in data through OptionMetrics. The code to pull data is written in such a way where certain configuration parameters can be changed to bring more data if needed (more data can be more options data over a longer period of time or more options). In addition, our team has developed code for the three numerical methods and has begun testing the code to make sure that all schemes have been properly implemented.

5.2 Code Demonstrations

While we are still testing the code to make sure outputs are correct, we have attached snippets of our code in the code listings with explanations below:

```
def solve_tridiagonal(lower: np.ndarray, diag: np.ndarray,
                      upper: np.ndarray, rhs: np.ndarray) ->
                          np.ndarray:
    n = len(diag)
```

```python

    if len(lower) != n - 1:
        raise ValueError(f"lower must have length {n-1}, got {len(lower)}")
    if len(upper) != n - 1:
        raise ValueError(f"upper must have length {n-1}, got {len(upper)}")
    if len(rhs) != n:
        raise ValueError(f"rhs must have length {n}, got {len(rhs)}")

    # Create copies to avoid modifying inputs
    c = np.zeros(n - 1)  # Modified upper diagonal
    d = np.zeros(n)      # Modified rhs

    # Forward elimination
    c[0] = upper[0] / diag[0]
    d[0] = rhs[0] / diag[0]

    for i in range(1, n - 1):
        denominator = diag[i] - lower[i-1] * c[i-1]
        c[i] = upper[i] / denominator
        d[i] = (rhs[i] - lower[i-1] * d[i-1]) / denominator

    # Last row (no upper diagonal element)
    d[n-1] = (rhs[n-1] - lower[n-2] * d[n-2]) / (diag[n-1] - lower[n-2] * c[n-2])

    # Back substitution
    x = np.zeros(n)
    x[n-1] = d[n-1]

    for i in range(n - 2, -1, -1):
        x[i] = d[i] - c[i] * x[i+1]

    return x
```

Listing 1: Thomas Algorithm for Tridiagonal System

The Thomas algorithm solves tridiagonal systems in O(n) time compared to O(n^3) for general Gaussian elimination. This efficiency is critical since our implicit schemes solve hundreds of such systems per time step.

```python
def implicit_step(self, V_n: np.ndarray, params: Dict[str, float], tau: float) -> np.ndarray:
        """Single backward time step using the implicit scheme (uses a linear system of equations)."""
        V_next = np.zeros_like(V_n)

        lower, diag, upper = self._build_tridiagonal_implicit(params)
```

```
        rhs = self._build_rhs_implicit(V_n, params, tau)

        # solving for interior nodes (excluding boundaries):
        V_next[1:-1] = solve_tridiagonal(lower, diag, upper, rhs)

        # applying boundary conditions at the new time level:
        V_next = self._apply_boundary_conditions(V_next, params, tau)

        return V_next
```

Listing 2: Implicit Finite Difference for Black-Scholes

This implementation demonstrates the complete workflow using various helper functions for implicit finite difference: grid generation, terminal condition setup, backward time-stepping with tridiagonal solves, and boundary condition enforcement. We set up the tridiagonal solver as its own module so that we could maintain consistency throughout the codebase, rather than developing a tridiagonal solver for each file

```
def craig_sneyd_scheme(self, V_n: np.ndarray, params:Dict) -> np.ndarray:
        N_v, N_S = V_n.shape
        V_bar = np.zeros_like(V_n)

        # S-direction sweep
        for j in range(1, N_v - 1):
            v_j = self.v[j]
            lower, diag, upper = self._build_tridiagonal_S(j, v_j, params)
            rhs = self._build_rhs_craig_sneyd_step_1(V_n, j, v_j, params)
            V_bar[j, 1:-1] = solve_tridiagonal(lower, diag, upper, rhs)

        # Copy boundary conditions from V_n to V_bar
        V_bar[0, :] = V_n[0, :]
        V_bar[-1, :] = V_n[-1, :]
        V_bar[:, 0] = V_n[:, 0]
        V_bar[:, -1] = V_n[:, -1]

        # v-direction sweep
        V_star = np.zeros_like(V_n)
        for i in range(1, N_S - 1):
            S_i = self.S[i]
            lower, diag, upper = self._build_tridiagonal_v(i, S_i, params)
            rhs = self._build_rhs_craig_sneyd_step_2(V_bar,
```

```
                i, S_i, params)
            V_star[1:-1, i] = solve_tridiagonal(lower, diag, upper, rhs)

        # Copy boundary conditions from V_bar to V_star
        V_star[0, :] = V_bar[0, :]
        V_star[-1, :] = V_bar[-1, :]
        V_star[:, 0] = V_bar[:, 0]
        V_star[:, -1] = V_bar[:, -1]

        # S-direction correction
        V_tilde = np.zeros_like(V_n)
        for j in range(1, N_v - 1):
            v_j = self.v[j]
            lower, diag, upper = self._build_tridiagonal_S(j, v_j, params)
            rhs = self._build_rhs_craig_sneyd_step_3(V_star, V_n, j, v_j, params)
            V_tilde[j, 1:-1] = solve_tridiagonal(lower, diag, upper, rhs)

        # Copy boundary conditions from V_star to V_tilde
        V_tilde[0, :] = V_star[0, :]
        V_tilde[-1, :] = V_star[-1, :]
        V_tilde[:, 0] = V_star[:, 0]
        V_tilde[:, -1] = V_star[:, -1]

        # Step 4: v-direction correction
        V_next = np.zeros_like(V_n)
        for i in range(1, N_S - 1):
            S_i = self.S[i]
            lower, diag, upper = self._build_tridiagonal_v(i, S_i, params)
            rhs = self._build_rhs_craig_sneyd_step_4(V_tilde, V_star, i, S_i, params)
            V_next[1:-1, i] = solve_tridiagonal(lower, diag, upper, rhs)

        # Copy boundary conditions from V_tilde to V_next
        V_next[0, :] = V_tilde[0, :]
        V_next[-1, :] = V_tilde[-1, :]
        V_next[:, 0] = V_tilde[:, 0]
        V_next[:, -1] = V_tilde[:, -1]

        return V_next
```

Listing 3: Craig-Sneyd ADI Scheme

The Craig-Sneyd scheme exemplifies the alternating direction approach: we solve all rows (S-direction), then all columns (v-direction), followed by two cor-

rection sweeps to achieve second-order accuracy. Each sweep requires N tridiagonal solves of size M, yielding O(NM) complexity per time step rather than O((NM)^3) for a full 2D implicit scheme. The correction steps update the mixed derivative term using intermediate values, which is the key difference from the simpler Douglas scheme.

```
def imex_euler_step(self, V_n: np.ndarray, params: Dict[str, float], tau: float) -> np.ndarray:
        """Single backward time step using first-order IMEX Euler."""
        V_next = np.zeros_like(V_n)

        lower, diag, upper = self._build_tridiagonal_implicit(params)
        rhs = self._build_rhs_imex(V_n, params, tau)

        V_next[1:-1] = solve_tridiagonal(lower, diag, upper, rhs)
        V_next = self._apply_boundary_conditions(V_next, params, tau)

        return V_next
```

Listing 4: IMEX Time-Stepping Structure for Jump-Diffusion

Using helper functions, this snippet shows 1 time-step using the IMEX scheme. The helper functions will split the PIDE operator into diffusion (local, implicit) and jump integral (nonlocal, explicit) components. This avoids inverting dense matrices that would arise from implicit treatment of the integral term while maintaining stability through implicit diffusion. The jump integral is computed via Gaussian quadrature over the jump size distribution.

5.3 Division of Labor

The remaining components of the project include incorporating Lavenberg-Marquardt calibration to optimize various parameters. We also need to develop code for the different models. Pascal will work on the calibration step, while Sarang and Mikey will work on building visualizations for the final presentation. Pascal will build the model for the Black-Scholes Model, Sarang will build the model for the Heston Model, and Mikey will build the Jump-Diffusion Model, as these assignments correspond to the numerical methods each person built out. Work on the final report and presentation will be split equally after code has been developed and properly calibrated.

References

[1] Rama Cont and Ekaterina Voltchkova. A finite difference scheme for option pricing in jump diffusion and exponential Lévy models. *SIAM Journal on Numerical Analysis*, 43(4):1596–1626, 2005.

[2] Yiran Cui, Sebastian del Baño Rollin, and Guido Germano. Full and fast calibration of the Heston stochastic volatility model. *European Journal of Operational Research*, 263(2):625–638, 2017.

[3] Karel J. in 't Hout and Sophine Foulon. ADI finite difference schemes for option pricing in the Heston model with correlation. *International Journal of Numerical Analysis and Modeling*, 7(2):303–320, 2010.

[2] [3] [1]

Artificial Intelligence (AI) Statement: AI has been used thus far in the project for the purpose of ideating (specifically conducting feasibility analysis for ideas the group had come up with) and for enhancing the readability of the literature review to better communicate the complicated material covered in the three papers. While coding, AI tools such as Claude, ChatGPT, and Cursor were used for debugging and adding documentation. An AI-powered citation generation website was used for the above references.

A Literature Review

A Heston stochastic model is a method to price options where both the price of the underlying stock and its volatility evolve randomly through time. It is a difficult model to calibrate as it is framed over 5 variables (dimensions) which are not entirely independent. The calibration approach proposed by the authors is a derived form of analytical gradient and natural optimization. They frame the problem as a nonlinear least-squares problem (comparing modeled price with actual price) by deriving a numerically continuous representation of the Heston characteristic function which avoids discontinuities, and formulating an analytical gradient which combined with Levenberg-Marquard (LM) algorithms enables fast gradient-based optimization. This issue is important because in option pricing time is a determinant factor for success as well as accuracy: the Heston model calibration landscape is irregular (proposed solutions don't seem to converge rapidly).

In 't Hout and Foulon (2010) analyze numerical methods for solving the two-dimensional Heston partial differential equation, which arises when modeling stochastic volatility with nonzero correlation between the asset price and its variance. Unlike the Black Scholes equation, the Heston PDE includes a mixed spatial derivative term that complicates stability and efficiency of standard finite difference methods. After discretizing space on nonuniform grids concentrated near the strike and low variance region, the PDE becomes a large stiff system of ordinary differential equations. The authors study several Alternating Direction Implicit schemes that split the operator into directional components, reducing computational complexity while maintaining accuracy. Through stability and convergence experiments, they show that appropriately designed ADI schemes achieve unconditional stability and second order convergence with computational cost proportional to the number of grid points. This paper is directly relevant to our project because it provides a practical and theoretically grounded framework for implementing efficient PDE solvers for the Heston model. While the authors focus on numerical stability and convergence of specific schemes, we incorporate these methods into a broader comparative framework that evaluates performance across models and links numerical accuracy to empirical calibration outcomes.

Cont and Voltchkova (2005) present a finite difference method for solving parabolic partial integro-differential equations (PIDEs) that arise in option pricing when asset dynamics follow Lévy processes or jump-diffusion models. Unlike pure diffusion models, Lévy processes incorporate discontinuous jumps in asset prices, offering greater flexibility in capturing market phenomena such as volatility smiles and heavy-tailed return distributions. The authors propose an explicit-implicit (IMEX) time-stepping scheme that treats the differential component implicitly via standard finite difference discretization while handling the nonlocal integral term explicitly, thereby avoiding inversion of dense matrices. They establish localization error bounds for truncating the spatial domain and integral kernel, demonstrating exponential decay under integrability conditions on the Lévy measure. The paper rigorously analyzes the scheme's consistency,

stability, and convergence using the viscosity solution framework, proving unconditional stability and monotonicity properties that ensure discrete comparison principles and arbitrage-free approximations. Convergence rates are derived under additional smoothness assumptions, and numerical experiments validate the method's accuracy and efficiency for both the Merton and variance Gamma models, including European and barrier options with smooth and nonsmooth payoffs.

Our project builds a unified option pricing framework that systematically compares numerical solution methods across multiple models and validates them through calibration to real market data. While Cui et al. (2017) focus on fast Heston calibration using analytical gradients, In 't Hout and Foulon (2010) analyze ADI scheme stability for the Heston PDE, and Cont and Voltchkova (2005) develop IMEX methods for jump-diffusion PIDEs, our work synthesizes these approaches into a comprehensive comparative study. We implement explicit, implicit, and Crank-Nicolson finite difference schemes for Black-Scholes, ADI methods for the two-dimensional Heston stochastic volatility model, and IMEX schemes for Merton or Kou jump-diffusion PIDEs, then rigorously evaluate each method's stability, convergence, and computational efficiency. The project applies linear systems solvers for tridiagonal and block-structured matrices at each time step, eigenvalue analysis for von Neumann stability conditions, numerical differentiation for computing Greeks and objective function gradients, and Levenberg-Marquardt nonlinear optimization for calibrating model parameters to OptionMetrics implied volatility surfaces. Unlike prior work that optimizes individual components, our contribution is a complete end-to-end pipeline that evaluates the interplay between numerical accuracy and empirical model fit, demonstrating how discretization choices affect both computational performance and calibration quality when fitted to real market data.